

End-to-End Machine Learning Pipeline

Project	Nightly Price Prediction for Airbnb Listings — New York City
Dataset	NYC Airbnb Open Data 2019 (~49,000 listings, 16 features)
Target Variable	price — nightly listing price in USD
Best Model	XGBoost (tuned, GridSearchCV)
Final R²	0.6480
Final MAE	\$44.78 per night
Final MAPE	30.76%
Author	Data Science Team
Year	2024

Table of Contents

1. Project Overview & Pipeline	3
2. Imports & Configuration	4
3. Data Loading & Description	4
4. Data Preprocessing & Cleaning	5
5. Exploratory Data Analysis (EDA)	6
6. Feature Engineering	8
7. Feature Selection	9
8. Model Training & Evaluation	10
9. Hyperparameter Tuning	12
10. Saving Model Artefacts	13
11. Interactive Price Prediction	13
12. Conclusions & Key Findings	14

1 Project Overview & Pipeline

This report documents a complete, reproducible machine learning pipeline for predicting the nightly listing price of Airbnb properties in New York City. The dataset used is the publicly available NYC Airbnb Open Data 2019, containing approximately 49,000 listings across five boroughs with 16 raw features. The pipeline spans ten stages — from raw data ingestion through exploratory analysis, feature engineering, model comparison, and hyperparameter tuning — ultimately producing a serialised prediction system ready for deployment.

End-to-End Pipeline Flow

Stage	Step	Description
1	Raw Data	Load CSV, inspect shape, dtypes, missing values
2	Preprocessing	Drop irrelevant cols, impute, remove outliers, encode
3	EDA	Univariate, bivariate, multivariate, geographic analysis
4	Feature Engineering	Create log transforms, ratios, binary flags, interaction terms
5	Feature Selection	Mutual Information + Random Forest importance filtering
6	Model Training	9 algorithms, 5-fold cross-validation, full metrics
7	Hyperparameter Tuning	GridSearchCV on best model (XGBoost)
8	Save Artefacts	Persist model, scaler, and feature list to disk
9	Prediction	Interactive price predictor from user inputs

Note: Model performance note: Airbnb price prediction is inherently noisy because hosts set prices based on factors not captured in this dataset (amenities, photos, décor, dynamic pricing tools). R^2 values in the 0.55–0.65 range are typical and considered good for this domain.

2 Imports & Configuration

Before analysis begins, a clean reproducible environment is established. All libraries are imported once at the top to make dependencies visible and avoid hidden re-imports. A fixed random seed (**RANDOM_STATE = 42**) ensures every run produces identical results. Plotting defaults are set globally so every chart follows a consistent visual style.

Global Constants

Constant	Value	Purpose
RANDOM_STATE	42	Reproducibility seed for all stochastic operations
TEST_SIZE	0.20	20% held-out test set
CV_FOLDS	5	5-fold cross-validation
PALETTE	'tab10'	Matplotlib/Seaborn colour palette

Optional libraries XGBoost and LightGBM are detected gracefully at import time so the notebook runs even if those packages are absent. LightGBM was not installed in the execution environment, so only XGBoost was available as a boosting alternative. All core scikit-learn modules — preprocessing, model selection, metrics, and feature selection — were loaded successfully.

3 Data Loading & Description

The pipeline auto-detects the dataset file by searching a list of common filenames in the working directory. If none is found it downloads the public NYC Airbnb 2019 dataset automatically via a URL. On successful load, the script reports shape, memory usage, and column types.

Total Rows	Columns	Memory	Price Range
48,895	16	24.6 MB	\$0–\$10k

Dataset Dictionary

Column	Type	Description
id	int	Unique listing identifier
name	str	Free-text listing title
host_id	int	Unique host identifier

host_name	str	Host first name
neighbourhood_group	str	Borough (Manhattan, Brooklyn, Queens, Bronx, Staten Island)
neighbourhood	str	Specific neighbourhood within borough
latitude	float	GPS latitude
longitude	float	GPS longitude
room_type	str	Entire home/apt, Private room, Shared room
price	int	TARGET — nightly price in USD
minimum_nights	int	Minimum booking duration
number_of_reviews	int	Total reviews received
last_review	date	Date of most recent review
reviews_per_month	float	Average monthly reviews
calculated_host_listings_count	int	Total listings this host has
availability_365	int	Days available per year

Missing Value Audit

Column	Missing Count	Missing %	Action
reviews_per_month	10,052	20.56%	Fill with 0 (zero reviews → 0/month)
last_review	10,052	20.56%	Drop column (date string, not predictive)
name	16	0.03%	Drop column (free-text, high cardinality)
host_name	21	0.04%	Drop column (PII, not predictive)

Price Summary Statistics

Metric	Value
Minimum price	\$0 (11 invalid rows to be removed)
Maximum price	\$10,000
Mean price	\$152.72
Median price	\$106.00
Zero-price rows	11 (data errors)

4 Data Preprocessing & Cleaning

Raw data is rarely analysis-ready. The preprocessing stage addresses three categories of issues: irrelevant/leaky columns, missing values, and outliers, followed by categorical encoding.

4.1 Column Dropping

Five columns were removed: **id**, **name**, **host_id**, **host_name**, and **last_review**. These are unique identifiers or free-text fields that carry no generalisable signal, or date strings that would require special parsing. The dataset shrank from 16 to 11 columns while retaining all 48,895 rows.

4.2 Missing Value Imputation

reviews_per_month is null whenever **number_of_reviews** == 0. This is structured missingness — a listing with zero reviews has zero reviews per month. We fill with 0 (not the column mean) because using the mean would incorrectly suggest these listings are actively reviewed. After imputation, 10,052 nulls were resolved and the total NaN count dropped to 0.

4.3 Outlier Removal

Rule	Threshold	Rows Removed	Rationale
Price = \$0	price == 0	11	Data errors / test listings
Price outliers	> 99.5th pct (~\$1,000)	239	Statistically rare; distort training
Minimum nights cap	> 99th pct (40 nights)	483	Monthly/annual rentals — different market

Metric	Value
Rows before cleaning	48,895
Rows removed total	733
Rows after cleaning	48,162
Retention rate	98.5%
Price range (clean)	\$10 – \$1,000

4.4 Categorical Encoding

Column	Strategy	Cardinality	Reason
--------	----------	-------------	--------

neighbourhood_group	One-Hot Encoding	5 boroughs	Avoids ordinal ranking artefacts
room_type	One-Hot Encoding	3 types	Avoids ordinal ranking artefacts
neighbourhood	Label Encoding	221 values	OHE would create 221 sparse columns

After encoding, the dataset expanded to **17 columns** (from 11), all numeric. Boolean dummy columns were cast to int32.

5 Exploratory Data Analysis (EDA)

EDA answers the fundamental question: *what patterns exist in the data, and do they make business sense?* Analysis is structured at three levels of complexity: univariate, bivariate, and multivariate.

5.1 Univariate Analysis — Target Variable (Price)

The raw price distribution is heavily right-skewed (skewness = **2.95**). Most listings cluster in the \$50–\$200 range with a long tail extending to \$1,000. Applying a $\log(\text{price} + 1)$ transformation reduces skewness dramatically to **0.33**, bringing the distribution close to normal — a requirement for linear models and beneficial for tree-based models as well.

Metric	Raw Price	$\log(\text{Price} + 1)$
Skewness	2.95	0.33
Distribution shape	Right-skewed (long tail)	Approximately normal
Suitability for models	Poor for linear models	Good for all model types

5.2 Univariate Analysis — Categorical Features

Room Type	Count	Share
Entire home/apt	24,900	51.7%
Private room	22,118	45.9%
Shared room	1,144	2.4%

Borough	Listings	Share
Manhattan	21,220	44.1%
Brooklyn	19,870	41.3%
Queens	5,625	11.7%
Bronx	1,077	2.2%
Staten Island	370	0.8%

5.3 Bivariate Analysis — Price by Categorical Features

Price varies substantially by both room type and borough. Entire home/apt listings command more than double the median price of shared rooms. Manhattan listings have a median price 2.3× higher than Bronx listings.

Room Type	Median Price	Mean Price	Count
Entire home/apt	\$160	\$195	24,900
Private room	\$70	\$85	22,118
Shared room	\$45	\$68	1,144

Borough	Median Price	Mean Price	Count
Manhattan	\$150	\$179	21,220
Brooklyn	\$90	\$118	19,870
Queens	\$75	\$95	5,625
Staten Island	\$75	\$99	370
Bronx	\$65	\$86	1,077

Borough × Room Type Cross-Analysis (Median Price)

Borough	Entire home/apt	Private room	Shared room
Manhattan	~\$195	~\$95	~\$65
Brooklyn	~\$135	~\$65	~\$45
Queens	~\$110	~\$60	~\$35
Staten Island	~\$110	~\$55	~\$45
Bronx	~\$90	~\$55	~\$35

Key insight: Room type has the strongest price differentiation within each borough. Manhattan Entire home/apt has the highest median price; Bronx Shared room has the lowest.

5.4 Multivariate Analysis — Top Feature Correlations with Price

Pearson correlation analysis (on the encoded feature matrix) identifies room type and borough as the strongest price predictors. Longitude captures the east-west geographic gradient (west Manhattan = higher prices). Most numeric features show weak linear correlations, suggesting non-linear models will outperform linear ones.

Feature	Pearson r	Direction	Interpretation
room_type_Entire home/apt	+0.475	Positive	Whole-home listings cost more
room_type_Private room	-0.446	Negative	Private rooms cost less

neighbourhood_group_Manhattan	+0.287	Positive	Manhattan premium
longitude	-0.258	Negative	West (Manhattan) = higher price
neighbourhood_group_Brooklyn	-0.168	Negative	Brooklyn discount vs Manhattan
neighbourhood_group_Queens	-0.144	Negative	Queens discount
calculated_host_listings_count	+0.130	Positive	Pro operators charge more
availability_365	+0.118	Positive	More available = busier / higher price

5.5 Geographic Analysis

A scatter plot of 10,000 sampled listings (colour-coded by log price) reveals that high-price listings concentrate in lower Manhattan and Williamsburg (western Brooklyn). Entire home/apt listings (red) dominate the Manhattan core, while private rooms (blue) are distributed across all boroughs.

6 Feature Engineering

Feature engineering creates new input variables from existing data that better represent underlying patterns. Nine new features were added, increasing the column count from 17 to 27.

Feature	Formula	Rationale
log_price	$\log(\text{price} + 1)$	Normalises right-skewed target
log_min_nights	$\log(\text{minimum_nights} + 1)$	Skewed; log captures diminishing effect
log_num_reviews	$\log(\text{number_of_reviews} + 1)$	Compresses review count range
reviews_per_listing	$\text{num_reviews} / \max(\text{host_listings}, 1)$	Per-listing demand proxy
availability_rate	$\text{availability_365} / 365$	0–1 rate; easier generalisation
has_reviews	1 if num_reviews > 0 else 0	New/inactive listings behave differently
is_superhost_proxy	1 if host_listings > 3	Professional operators charge more
engagement_score	$\text{reviews_per_month} \times \text{availability_rate}$	Combined demand signal
lat_lon_product	$\text{latitude} \times \text{longitude} $	Captures joint location effect

Why Log-Transform the Target?

When a model is trained on $\log(\text{price})$, it minimises squared errors in log-space. This means a \$10 error on a \$50 listing is penalised equally to a \$10 error on a \$500 listing — a more realistic business objective than minimising absolute dollar errors, which would cause the model to over-fit on expensive outliers. Final predictions are recovered via the inverse transform: **$\text{pred_price} = \exp(\text{pred_log_price}) - 1$** .

***Note:** `min_spend_equiv` ($= \text{price} \times \text{minimum_nights}$) is explicitly excluded from the feature matrix to prevent data leakage — it contains the target variable (`price`) implicitly.*

7 Feature Selection

Feature selection removes columns that contribute noise rather than signal. Too many features cause overfitting and increase training time. Two complementary methods were applied and their results combined.

7.1 Methods

Mutual Information (MI): Measures how much knowing a feature reduces uncertainty about the target. Captures non-linear relationships. Decision rule: features with $MI < 0.01$ are dropped.

Random Forest Importance: A quick 100-tree RF model is trained and `feature_importances_` (mean decrease in impurity) provides a tree-based ranking. Features in the bottom 10th percentile by RF importance are dropped.

Final selection: The union of features retained by both methods. This conservative approach ensures we keep features important by at least one criterion.

7.2 Selection Results

Features in	MI threshold	RF threshold	Features out
23	0.01	10th pct	21

Three features dropped by MI (score < 0.01):

- neighbourhood_group_Bronx
- engagement_score
- neighbourhood_group_Staten Island

7.3 Top Features by Mutual Information Score

Feature	MI Score	RF Importance
room_type_Entire home/apt	0.2904	0.6424
room_type_Private room	0.2507	0.0173
neighbourhood_enc	0.2180	0.0883
longitude	0.1673	0.0621
calculated_host_listings_count	0.1658	0.0094
latitude	0.1316	0.0244

lat_lon_product	0.1505	0.0244
neighbourhood_group_Manhattan	0.0814	0.0542
log_min_nights	0.0556	0.0173
reviews_per_listing	0.0607	0.0050

8 Model Training & Evaluation

Nine regression algorithms were evaluated using an 80/20 train/test split (fixed seed 42) and 5-fold cross-validation on the training set. Linear models received RobustScaler preprocessing; tree-based models were fed raw features. All metrics are computed on the held-out test set.

8.1 Models Evaluated

Model	Type	Key Hyperparameters
Linear Regression	Linear (baseline)	—
Ridge	Linear + L2 reg.	$\alpha = 1.0$
Lasso	Linear + L1 reg.	$\alpha = 0.01$
ElasticNet	Linear + L1 + L2	$\alpha = 0.01$, l1_ratio = 0.5
Decision Tree	Non-linear, single tree	max_depth = 10
Extra Trees	Ensemble (bagging)	300 trees, max_depth = 15
Random Forest	Ensemble (bagging)	300 trees, max_depth = 15
Gradient Boosting	Ensemble (boosting)	300 trees, lr = 0.05
XGBoost	Gradient boosting (opt.)	500 trees, lr = 0.05

8.2 Evaluation Metrics

Metric	Description	Better when
R ²	Proportion of variance explained	Higher (max 1.0)
MAE (log)	Mean absolute error in log-price space	Lower
RMSE (log)	Root mean squared error in log-price	Lower
MAE (\$)	Mean absolute dollar error	Lower
MAPE (%)	Mean absolute percentage error	Lower

8.3 Full Results Table

Model	CV R ²	CV σ	Test R ²	MAE(log)	RMSE(log)	MAE(\$)	MAPE%
XGBoost	0.6398	0.0061	0.6462	0.2943	0.3946	\$44.93	30.87%

Gradient Boosting	0.6325	0.0051	0.6414	0.2979	0.3973	\$45.58	31.16%
Random Forest	0.6300	0.0054	0.6396	0.2974	0.3983	\$45.38	31.21%
Extra Trees	0.6282	0.0055	0.6368	0.2979	0.3998	\$45.45	31.25%
Decision Tree	0.5828	0.0037	0.5968	0.3167	0.4213	\$48.26	33.55%
Linear Regression	0.5382	0.0034	0.5478	0.3379	0.4461	\$50.56	35.64%
Ridge	0.5318	0.0032	0.5400	0.3415	0.4500	\$51.13	36.16%
ElasticNet	0.5213	0.0025	0.5284	0.3451	0.4556	\$51.39	36.54%
Lasso	0.5114	0.0020	0.5185	0.3482	0.4603	\$51.67	36.91%

8.4 Key Observations

- Ensemble tree models significantly outperform linear models (+0.09 R^2 advantage), confirming non-linear relationships in pricing.
- XGBoost achieves the best Test R^2 (0.6462), MAE(\$44.93), and MAPE(30.87%).
- All models show $CV\ R^2 \approx Test\ R^2$, indicating no meaningful overfitting.
- Linear models (Ridge, Lasso, ElasticNet) form a performance floor at $\sim 0.52\ R^2$, representing the amount of variance explainable by linear relationships alone.
- The 0.10-point R^2 gap between linear and ensemble models quantifies the value of capturing interaction effects (e.g., borough $\times$ room type).

8.5 Feature Importance — XGBoost

Feature	Importance	Share of Total
room_type_Entire home/apt	0.4875	48.8%
room_type_Private room	0.1828	18.3%
neighbourhood_group_Manhattan	0.1625	16.3%
room_type_Shared room	0.0630	6.3%
longitude	0.0117	1.2%

All other features combined	~0.0925	9.1%
-----------------------------	---------	------

The top three features (room type + Manhattan flag) account for over 83% of the model's predictive power, confirming that property type and location dominate Airbnb pricing.

9 Hyperparameter Tuning

GridSearchCV with 5-fold cross-validation was applied to XGBoost (the best-performing model) to find optimal hyperparameters. The grid comprised 24 combinations × 5 folds = 120 total model fits, completed in 142.4 seconds.

9.1 Search Grid

Parameter	Values Searched	Effect
n_estimators	[300, 500]	More trees = better fit, slower training
max_depth	[5, 6, 7]	Deeper = more complex, more overfit risk
learning_rate	[0.03, 0.05]	Lower = needs more trees, better generalisation
subsample	[0.8, 0.9]	Row sampling — reduces overfitting
colsample_bytree	[0.8]	Feature sampling per tree

9.2 Best Parameters Found

Parameter	Best Value
n_estimators	500
max_depth	7
learning_rate	0.03
subsample	0.8
colsample_bytree	0.8
Best CV R ²	0.6417

9.3 Pre vs Post Tuning Comparison

Metric	Before Tuning	After Tuning	Improvement
Test R ²	0.6462	0.6480	+0.0018
MAE (log)	0.2943	0.2929	-0.0014
MAE (\$)	\$44.93	\$44.78	-\$0.15
MAPE (%)	30.87%	30.76%	-0.11 pp

Tuning provided marginal but consistent improvement across all metrics. The small delta indicates the default hyperparameters were already well-chosen. In production, wider search via RandomizedSearchCV or Bayesian optimisation (Optuna) would be recommended.

10 Saving Model Artefacts

Three artefacts are serialised to disk using joblib, together forming a complete, self-contained prediction system. Any analyst can load them and produce identical predictions without access to the original training data.

File	Contents	Purpose
models/best_model.pkl	Tuned XGBoost estimator	Ready-to-predict model object
models/scaler.pkl	RobustScaler (21 features)	Scale new inputs for linear models
models/feature_names.pkl	Ordered list of 21 feature names	Align new data to training columns
models/results_summary.pkl	Full comparison DataFrame	Audit trail of all model scores

Final Model Performance

R ²	MAE (\$)	MAPE	Model
0.6480	\$44.78	30.76%	XGBoost

11 Interactive Price Prediction

Section 10 implements an interactive command-line predictor that uses the saved model artefacts to estimate a listing's nightly price from user-supplied inputs. All three artefacts are reloaded from disk, confirming the save-and-load cycle works correctly.

11.1 Input Fields & Defaults

Input Field	Default Value	Notes
latitude	40.7282	Central Manhattan latitude
longitude	-73.9942	Central Manhattan longitude
minimum_nights	3	Median from training set
number_of_reviews	23	Mean from training set
reviews_per_month	1.37	Mean from training set
host_listings_count	1	Single-listing host
availability_365	225	Median from training set
room_type	Private room	Case-insensitive matching
borough	Brooklyn	Case-insensitive matching

11.2 Sample Prediction Output

Using default inputs (Brooklyn, Private room), the model produced:

Predicted Price	Low Estimate	High Estimate	Confidence
\$91.45	\$63.31	\$119.58	±30.8%

The confidence interval is derived from the model's MAPE (30.76%), applied symmetrically around the point estimate. This reflects the inherent variability in Airbnb pricing — factors such as amenities, photos, and dynamic pricing tools are not captured in the dataset.

12 Conclusions & Key Findings

12.1 Primary Findings

- **Room type is the single most important predictor**, accounting for nearly 73% of XGBoost's predictive power. Entire home/apt listings command a median premium of ~\$90/night over private rooms.
- **Location (borough and neighbourhood) is the second most important factor**. Manhattan commands a \$60 median premium over Brooklyn. Geographic coordinates (lat/lon) capture within-borough price gradients that discrete borough labels miss.
- **Ensemble models substantially outperform linear baselines**. The 0.09 R^2 gap demonstrates significant non-linear and interaction effects that linear regression cannot capture.
- **Log-transforming the target is essential**. The transformation reduced skewness from 2.95 to 0.33, enabling fair penalisation of errors across all price ranges.
- **The final XGBoost model generalises well**. The negligible gap between CV R^2 (0.6398) and Test R^2 (0.6462) confirms no meaningful overfitting.

12.2 Model Performance Summary

Metric	Value	Interpretation
Test R^2	0.6480	64.8% of price variance explained — good for noisy domain
MAE (\$)	\$44.78	Average prediction error of \$44.78 per night
MAPE	30.76%	Average relative error of ~31% — acceptable for Airbnb
CV R^2	0.6417	Stable cross-validation score — no overfitting

12.3 Limitations

- The dataset does not include listing photos, amenity counts, or cancellation policy — all known drivers of price.
- Neighbourhood label encoding imposes an arbitrary ordinal ranking across 221 neighbourhoods; target-mean encoding would be more appropriate in production.
- The 2019 dataset predates COVID-19 disruptions; the model may not generalise to post-pandemic pricing dynamics.
- Dynamic pricing (nightly price changes) is not captured; the model predicts a single representative price per listing.

12.4 Recommended Next Steps

- Integrate listing amenity data from the Airbnb API (WiFi, pool, parking) to improve R^2 toward 0.75+.
- Replace neighbourhood label encoding with target-mean encoding to better capture the neighbourhood price gradient.
- Implement SHAP (SHapley Additive exPlanations) values for per-prediction interpretability.
- Deploy the model as a REST API (FastAPI + Docker) with the three saved artefacts.
- Re-train annually on refreshed data; monitor MAPE drift in production.

Report generated from the Airbnb NYC Price Prediction notebook. All results are fully reproducible with `RANDOM_STATE = 42`.